ADS205 Decision, Computation and Language

Lecture 1 – Introduction and Regular Language

Dr Chunchuan Lyu and Dr Xinhong Chen



Xi'an Jiaotong-Liverpool University
西交利物浦大学

Decision, Computation and Language

What this module is ***not*** About

- Study a bunch of simplified models of ***computation***, because it is hard to understand them.

  DFA/NFA,PDA,TMs

- Make abstract definition of ***language***, because it is cool.

  Language is a set of strings

- ***Deciding*** whether a string is in a language, because it is the most important ***problem*** in the world.

  Is $0 \in \{0,11,01\}$ true?



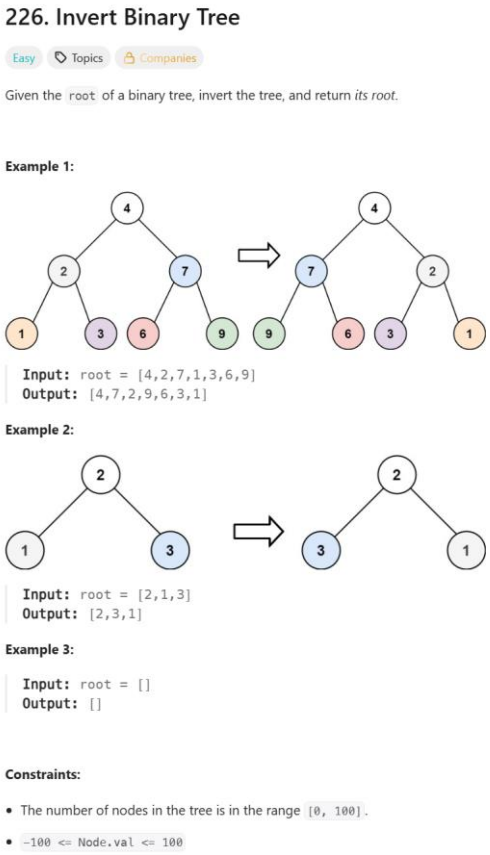
Decision, Computation and Language

What this module is About

Study the nature of computation

- What is computation?

  Algorithms



226. Invert Binary Tree

Easy Topics Companies

Given the `root` of a binary tree, invert the tree, and return *its root*.

Example 1:


```
Input: root = [4,2,7,1,3,6,9]
Output: [4,7,2,9,6,3,1]
```

Example 2:


```
Input: root = [2,1,3]
Output: [2,3,1]
```

Example 3:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 100]`.
- `-100 <= Node.val <= 100`



Decision, Computation and Language

What this module is About

Study the nature of computation

- What is computation?

  Algorithms, Physical Computation, Perception , etc.

Solve a second order Newtonian aero dynamics equations of a rigid body

v.s.





Decision, Computation and Language

What this module is About

Study the nature of computation

- What is computation? Algorithms, Physical Computation, Perception, etc.
- What can be computed?

Write algorithms, build physical devices, create life?



Decision, Computation and Language

What this module is About

Study the nature of computation

- What is computation? Algorithms, Physical Computation, Perception, etc.

- What can be computed? Write algorithms, build physical devices, create life.

- What can not be computed?

We need a highly simplified but fully specified definition of computation.

Now, ***"deciding whether a string is in a language"*** sounds lovely.



Decision, Computation and Language

What this module is About

Study the nature of computation

- What is computation?
- What can be computed?
- What can not be computed?

We need a highly simplified but fully specified definition of computation.

deciding whether a string is in a language

but why is this sufficient?

you need to stick to the first half, and this will be clear when you learned Turing machine.



What You will Gain

- Understanding the Limit of Computation

  There are problems that can not be solved

  No computation can decide whether any pieces of code implement a specified non-trivial function

  Bonus: what if my machine is probabilistic? (e.g., LLMs)



What You will Gain

- Mathematical Maturity

The ability to think clearly and engage in formal arguments. Formal in this module means that we ground terminologies in **set theoretical language** whenever we can. (not as formal as the Lean language)

Your math ability will be amplified by AI. If you know how to formalize your question, you can leave the proving part to AI. You still probably need to verify and definitely need to digest.



We have an AI problem in Education

DIVE BRIEF

AI can complete 'almost' all written assignments for MIT undergrads, report says

To adapt, faculty must be able to quickly update their curriculum without "multi-committee, year-long review processes," a university committee said.

So, we do not have marked homework ☺

AI Study Tip



Use AI to guide you in the new fields, explanation in your native language just makes you feel that you understand the concepts.
Ask AI to challenge you, not comforting you.

What are Expected of You?

- Follow the lecture/tutorial during the semester.

This is a math module, and it is probably harder than any modules that you have taken and would take.

Last year, the average is above 70, because there were take home assignments. But not any more ☺



What are Expected of You?

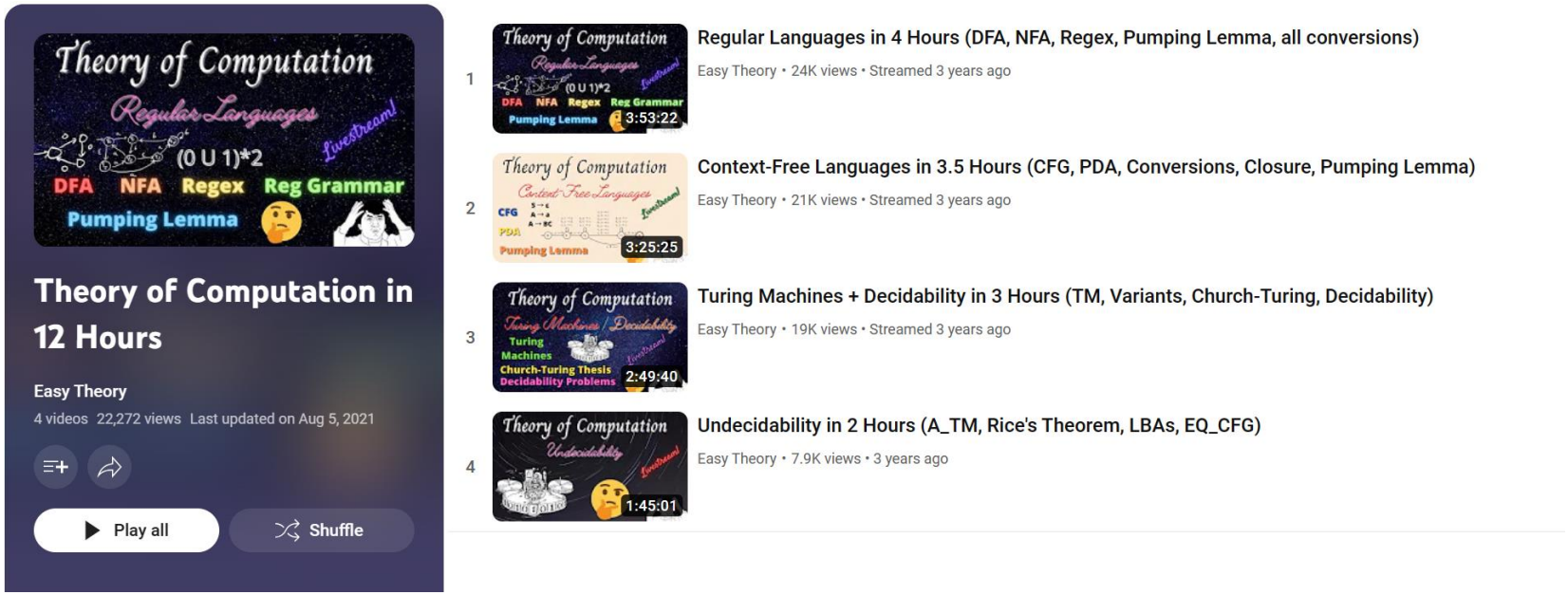
- Ask questions during the lecture.

You will help the lecturer and your classmates when you ask for clarification. Basic questions are actually better!

Asking question is never about showing off how smart you are, which is wasting other people's time. It is about genuine confusion.

Asking question makes the presenter feel better. It shows that audience are following and engaging in the topic being presented.

Highly Recommended Additional Resources



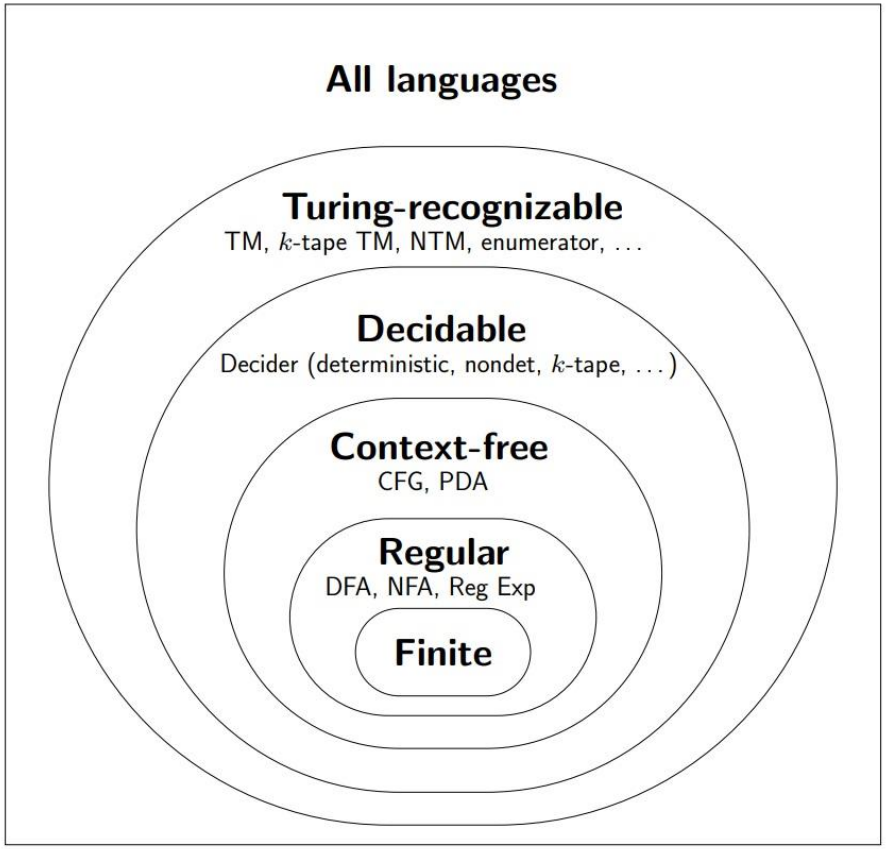
Theory of Computation in 12 Hours Detailed and intuitive explanation

https://github.com/ND-CSE-30151/spring-2026 Code to illustrate concepts

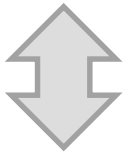
https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/ We covered a third of their topics

The Nature of Computation by Stephan Mertens, Cristopher Moore A very good textbook

Road Map



Model of Computation



Ability to decide whether a string is in a certain class of language

The more powerful the model, the more complicated the decision problem.

Logistics

- 2 20% in-class Tests using Learning Mall + 1 exam 60%.
- Weakly Lecture and Tutorial except Week 7 (Consolidation Week)
- Office Hours: ask anything



Assistant Professor-Dr Chunchuan Lyu

- Graduated from The University of Edinburgh and XJTLU

  Studied computational semantics but moved to unsupervised reinforcement learning (what an agent should do if no task is given?)

- Office hour: 15:15-17:15 Thursday at SD543 (or by appointment)
- Contact: chunchuan.lyu@xjtlu.edu.cn



Assistant Professor-Dr Xinhong Chen

- Graduated from City University of Hong Kong
  - Studied artificial intelligence and reinforcement learning
  - Research interests include *autonomous driving*, *causality analysis*, and *AI agents*

- Office hour: 10:00-12:00, 13:00-15:00 on Tuesday at SD346 (or by appointment)
- Contact: xinhong.chen@xjtlu.edu.cn



String and Languages: Formal

- A string is a finite sequence of symbols in Σ
- A language is a set of strings (finite or infinite)
- The empty string ε is the string of length 0
- The empty language ø is the set with no strings

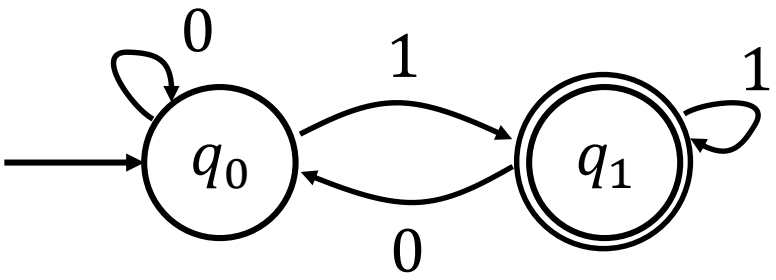
Examples:

- Finite alphabet Σ : {0,1}
- String : 10
- If the length is 0, the string is an empty string ε
- Language: {10, 010}, {}, {ε} , {0,00,000,…}



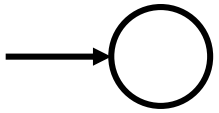
Finite Automata

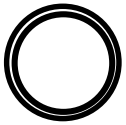
The simplest computation model



State: q_0, q_1

Transitions: $\xrightarrow{1}$, where "1" here stands for triggering conditions;

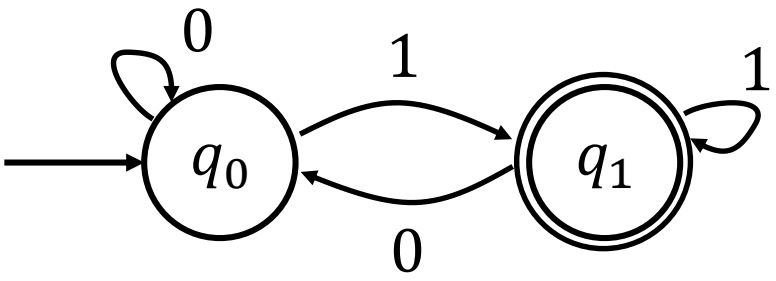
Start State:  , where the machine starts;

Accept / Final States:  , where the output is deemed **accepted**.



Finite Automata

The simplest computation model



Implicit assumptions:

- Machines have **fixed** memory. => ***There are finite states***
- Machines start in a specific state. => ***There must be a start state.***



Finite Automata: Formal

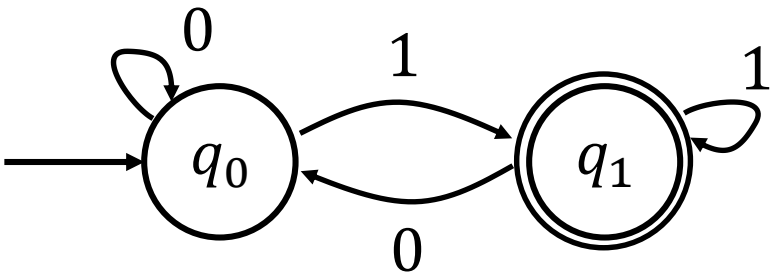
A finite automaton M is a 5-tuple vector $(Q, \Sigma, q_0, F, \delta)$:

- Q: a finite set of states
- Σ: a finite set of alphabet characters (i.e., all possible characters)
- $q_0 \in Q$: the start state
- $F \subseteq Q$: the set of accept/final states => *could be empty*
- $\delta: Q \times \Sigma \rightarrow Q$ transition functions

$\times$： The Cartesian Product



Finite Automata: Example



- Q: a finite set of states $Q = \{q_0, q_1\}$
- Σ: a finite set of alphabet characters $\Sigma = \{0,1\}$
- $q_0 \in Q$: the start state $q_0 = q_0$
- $F \subseteq Q$: the set of accept/final states $F = \{q_1\}$
- δ: $Q \times \Sigma \rightarrow Q$, δ(state, input) = next state $\delta(q_0, 0) = q_0, \ldots$

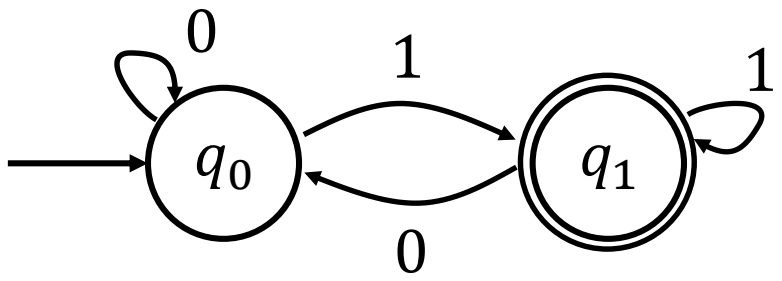
  or transition tables (with Σ as columns and Q as rows)

	0	1
q_0	q_0	q_1
q_1	q_0	q_1



Finite Automata

- A **string**: a finite sequence of characters in Σ
  - Example: 0100
  - If the length is 0, the string is an empty string ε.
- A string is the unit input for the finite automata
  - Same example: 0100
  - Starts in q_0, reads 0, stays in q_0
  - reads 1, transitions to q_1
  - reads 0, transitions to q_0
  - reads 0, stays in q_0 => finish reading, ends in q_0



Finite Automata Acceptance: Formal

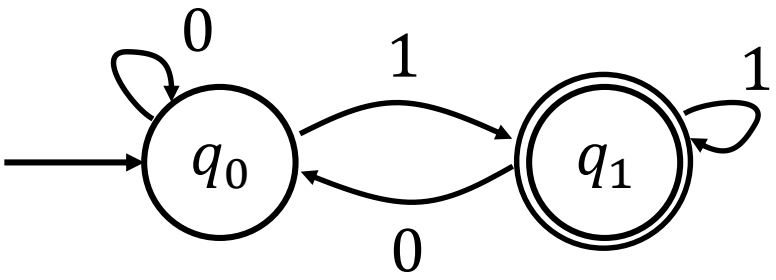
Defn: M accepts string $w = w_1 w_2 \ldots w_n$ each $w_i \epsilon \Sigma$ if there is a sequence of states $r_0, r_1, r_2, , \ldots, r_n \epsilon Q$ where:

- $r_0 = q_0$
- $r_i = \delta(r_{i-1}, w_i)$ for $1 \leq i \leq n$
- $r_n \epsilon F$

- For a given finite automata M and an input string w, if M reads **ALL** characters in w and **ends in an accept state**, we say $\boldsymbol{M}$ **accepts** $\boldsymbol{w}$.
- Otherwise, we say $\boldsymbol{M}$ **rejects** $\boldsymbol{w}$.



Finite Automata: Example

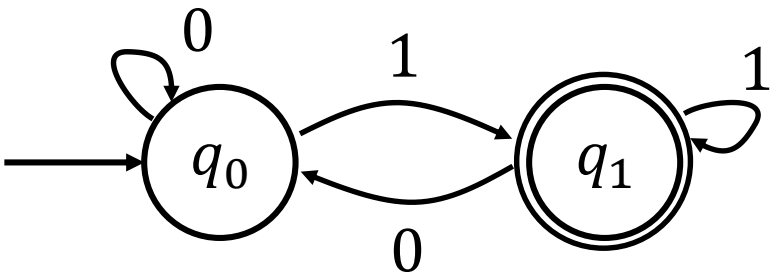


- $w = 011$
- starts in q_0, reads 0, stays in q_0
- reads 1, transitions to q_1
- reads 1, stays in q_1
- finish reading, ends in an accept state => ***M* accepts *w***.

	0	1
q_0	q_0	q_1
q_1	q_0	q_1



Finite Automata: Example



- $w = 0110$
- starts in q_0, reads 0, stays in q_0
- reads 1, transitions to q_1
- reads 1, stays in q_1
- reads 0, transitions to q_0
- finish reading, ends in a non-accept state => ***M* rejects *w***.

	0	1
q_0	q_0	q_1
q_1	q_0	q_1



Regular Language: Formal

Equivalent Notions

- $L(M) = \{w | M \text{ accepts } w\}$
- $L(M)$ is the language of M
- M recognizes $L(M)$

Defn: A language is regular if some finite automaton recognizes it.

- M recognize L if M accepts all w in L, and rejects all w not in L.
- Proving a language is regular => construct its corresponding finite automata.



Quick Review So Far

- A **string** is a sequence of characters;
- A **language** is a set of strings (finite or infinite);
- A finite automaton M is a 5-tuple vector $(Q, \Sigma, q_0, F, \delta)$:
- M **accepts** w , if M reads **ALL** characters in w and **ends in an accept state**
- *M* ***recognize*** *L* if *M* accepts all *w* in *L*, and rejects all *w* not in *L*.
- A language is **regular** if some finite automaton recognizes it.



Regular Operation: Formal

Why the name *regular*?

Indeed, finite automata are equivalent to regular expressions

Regular operations. Let A, B be languages:

- **Union:**

$$A \cup B = \{w | \, w \in A \text{ or } w \in B\}$$

- **Concatenation:**

$$A \circ B = \{xy | \, x \in A \text{ and } y \in B\} = AB$$

- **Star:**

$$A^* = \{x_1 \ldots x_k | \text{ each } x_i \in A \text{ for } k \geq 0\}$$

Note: $\varepsilon \in A^*$ always

Example: Let $A = \{\text{good, bad}\}$ and $B = \{\text{boy, girl}\}$.

- $A \cup B = \{\text{good, bad, boy, girl}\}$
- $A \circ B = AB = \{\text{goodboy, goodgirl, badboy, badgirl}\}$
- $A^* = \{\varepsilon, \text{good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, goodgoodbad, } \ldots\}$



Regular Expression: Formal

Regular Expression

An expression is <u>regular</u> if it is either atomic or composite of another regular expression

- Built from Σ, members $\Sigma, \emptyset, \varepsilon$ [Atomic]
- By using $\cup, \circ, *$ [Composite]

Regular operations. Let A, B be languages:

- **<u>Union</u>:**

$$A \cup B = \{w | \ w \in A \ \text{ or } \ w \in B\}$$

- **<u>Concatenation:</u>**

$$A \circ B = \{xy | \ x \in A \ \text{ and } \ y \in B\} = AB$$

- **<u>Star:</u>**

$$A^* = \{x_1 \ldots \ x_k | \ \text{each } x_i \in A \ \text{ for } \ k \geq 0\}$$

Note: $\varepsilon \in A^*$ always

Examples:

- $(0 \cup 1)^* = \Sigma^*$ gives all strings over Σ={0,1}
- $\Sigma^* 1$ gives all strings that end with 1
- $\Sigma^* 11 \Sigma^*$ = all strings that contain 11 = $L(M_1)$



Closure Properties of Languages: Formal

Goal: Show finite automata equivalent to regular expressions

We need some tools for analyzing a set of languages.

Closure of Binary Operation

- Set C is ***closed*** under $\oplus$: C x C-> C,

if $\forall A, B \in C$, $A \oplus B \in C$

Closure of Unary Operation

- Set C is ***closed*** under $\oplus$: C -> C,

if $\forall A \in C$, $\oplus A \in C$

- Regular language is a set of languages, regular operations can be examined for their closure property on regular language.



Closure Properties of Regular Language

- Say $A = \{w | w\ ends\ with\ 0\}$, $B = \{w | w\ ends\ with\ 1\}$, and $C = \{w | w\ ends\ with\ 0\ or\ 1\}$.
- Based on the description, we can easily observe that C is the union set of A and B, as it contains all the strings in A and B.

Other operations on languages

- From sets: union, intersection, complement, difference
- Unique on languages: concatenation, star, reverse, …

(string as element)



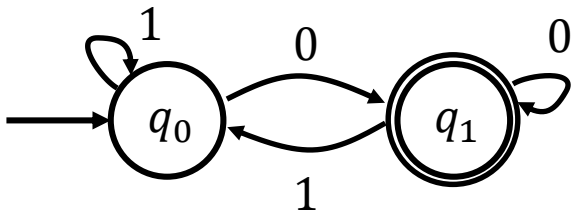
Regular Language is Closed under Union: Example

if A and B are both regular languages, so is $A \cup B$.

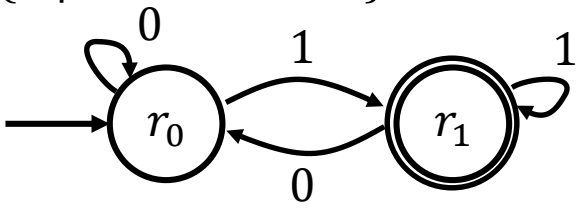
Let's examine one pair to warm up.

- We need an new automaton M for a pair of regular languages A and B
- If $w \in A \cup B = \{w | w \in A \; or \; w \in B\}$, $M \; accepts \; w$
- If $w \notin A \cup B$, $M \; rejects \; w$

$A = \{w | w \; ends \; with \; 0\}$

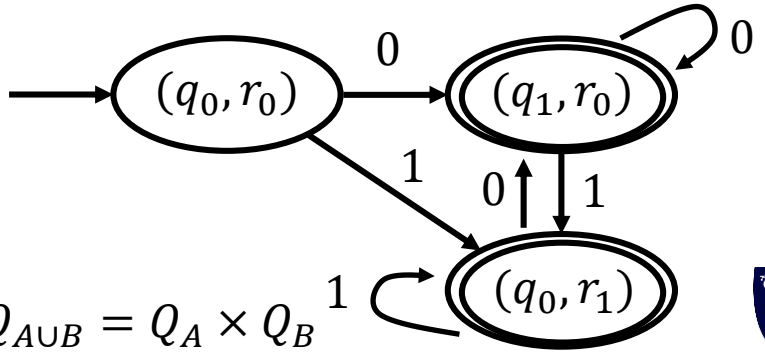


$B = \{w | w \; ends \; with \; 1\}$



$A \cup B = \{w | w \; ends \; with \; 0 \; or \; 1\}$.

"simulate" both machines simultaneously



$Q_{A \cup B} = Q_A \times Q_B$

*Note: The state set is the cartesian **product** of the two state sets.*

Regular Language is Closed under Union: Formal

Theorem: if A and B are both regular languages, so is $A \cup B$.

Equivalently, ***regular languages are closed under union*.**

Proof:

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A

$M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize B

We want a $M = (Q, \Sigma, \delta, q_0, F)$

that recognize $A \cup B$

Components of M:

$$Q = Q_1 \times Q_2$$
$$= \{(q_1, q_2) | q_1 \in Q_1 \text{ and } q_2 \in Q_2\}$$
$$q_0 = (q_1, q_2)$$
$$\delta((q, r), a) = (\delta_1(q, a), \delta_2(r, a))$$
$$F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$$

M simulates M1 and M2 in a product space, and accept in the union of acceptance states in production space.



Review of Today

- A **string** is a sequence of characters; A **language** is a set of strings;
- A finite automaton M is a 5-tuple vector $(Q, \Sigma, q_0, F, \delta)$:
- M **accepts** w , if M reads **ALL** characters in w and **ends in an accept state**
- *M* ***recognize*** *L* if *M* accepts all *w* in *L*, and rejects all *w* not in *L*.
- A language is **regular** if some finite automaton recognizes it.

- Regular operations on languages are union $\cup$, concatenation $\circ$, and star $*$.
- **Regular expressions**, are built from $\Sigma, \emptyset, and\ \varepsilon$ and regular operations $\cup,\circ,*$.
- Language **closure** properties are defined over **a set of languages.**
- Regular languages are **closed** under Union. To be continued



Q & A

① Does a finite automaton need to have at least one state?

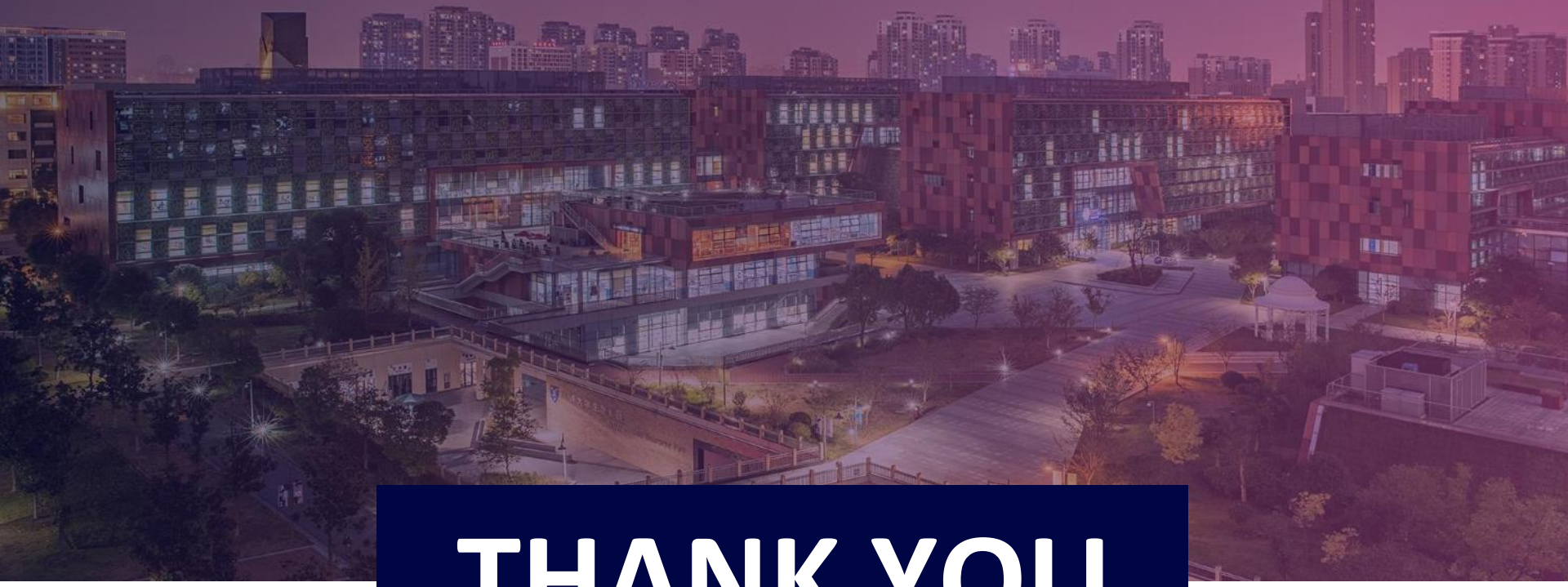
② What if we read a character but there is no corresponding transition?

③ Can we read a character but conduct no transition?

④ Can the empty string be accepted by a finite automata?

⑤ Is an empty language regular?





THANK YOU

Xi'an Jiaotong-Liverpool University
西交利物浦大學

XJTLU | SCHOOL OF FILM AND TV ARTS